

Dataset & Document Extraction — Technical Overview

1. Project Context

We are developing an **industry-oriented Insurance RAG AI Assistant**.

The system will answer questions using evidence from real insurance policy documents. Therefore, the quality of the RAG system depends heavily on how accurately we convert the original policy PDFs into a structured, retrieval-ready representation.

Our initial corpus contains **5 insurance products from 5 different insurers**:

ID	Insurer	Product
INS-001	HDFC ERGO	my:Optima Secure
INS-002	Niva Bupa	ReAssure 2.0
INS-003	Care Health Insurance	Ultimate Care
INS-004	ICICI Lombard	Elevate
INS-005	Star Health	Star Comprehensive

The selected documents are **real publicly available policy documents**, not synthetic documents.

2. Why We Selected Real Policy Documents

The objective isn't just to demonstrate basic RAG.

We want to test whether our system can work with **real-world heterogeneous insurance documentation**.

The five PDFs can differ in:

- number of pages
- section structure
- heading styles
- numbering conventions
- terminology
- page layouts
- tables
- clauses
- exclusions
- waiting periods
- cross-page content

- formatting

If we created synthetic documents ourselves, we would already control their structure.

That would make the extraction and chunking problem artificially easy.

Therefore:

Real policy documents are our primary corpus because document variability itself is one of the engineering problems we want to solve.

Synthetic data may later be used for controlled testing, but it is not our primary knowledge source.

3. What Is the Actual Dataset?

A very important distinction:

The dataset is NOT:

5 PDFs → one huge text file

and it is NOT:

5 PDFs → LLM summaries

Instead, we maintain three conceptual levels:

```
SOURCE DATA
  ↓
Structured Document Data
  ↓
Chunk Dataset
```

Source Data

The original PDFs.

Structured Document Data

A machine-readable representation of what exists inside each PDF.

Chunk Dataset

The later retrieval-oriented representation created from the structured documents.

4. Why We Need an Intermediate Representation

This is the most important part of our current design.

We cannot directly do:

PDF → chunks

because the PDFs contain structural information that can be lost during extraction.

For example:

```
Page 20
  Heading
  Paragraph
  Table

Page 21
  continuation of paragraph
  new section
```

If we immediately convert everything into plain text, we may lose:

- page boundaries
- layout
- table relationships
- reading order
- heading relationships
- source location
- cross-page continuity

Therefore our pipeline is:

```
PDF
↓
Extraction
↓
Intermediate Structured Representation
↓
Validation
↓
Smart Chunking
↓
Chunk Validation
↓
Embeddings
```

5. How We Will Extract the PDFs

We will process **each PDF independently**.

```
HDFC PDF
  ↓
HDFC structured representation

Niva Bupa PDF
  ↓
Niva Bupa structured representation

Care PDF
  ↓
Care structured representation

ICICI PDF
  ↓
ICICI structured representation

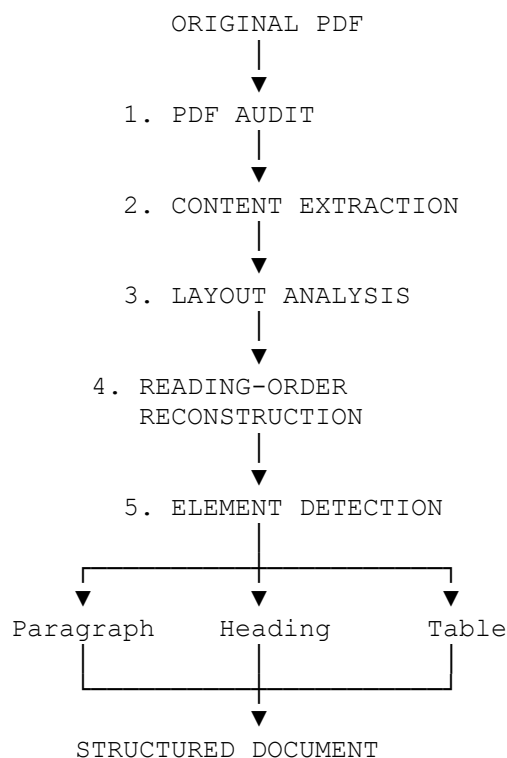
Star PDF
  ↓
Star structured representation
```

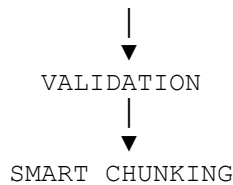
We will **not combine all 250+ pages into one giant JSON**.

The corpus-level registry will simply tell the system which documents belong to the corpus.

6. Extraction Pipeline

Our extraction pipeline will work in stages.





7. Stage 1 — Document Audit

Before extraction, we automatically inspect each PDF.

We want to know:

- total pages
- whether pages contain extractable text
- whether some pages are image/scanned pages
- document metadata
- possible tables
- layout characteristics
- potential extraction problems

Example:

```
{
  "document_id": "STAR_COMP_V1",
  "page_count": 59,
  "text_pages": 58,
  "image_pages": 1,
  "status": "AUDITED"
}
```

This gives us an early understanding of the document instead of blindly processing it.

8. Stage 2 — Content Extraction

We extract the actual content from each page.

But we don't want only:

```
"page 20 text..."
```

We want individual **document elements**.

For example:

```
Page 20
├── Heading
└── Paragraph
```

└ Paragraph
└ Table

Each element should retain its source location.

9. Stage 3 — Layout Analysis

PDFs are visual documents, not simple text files.

A page could be:

Single column

Heading

Paragraph
Paragraph
Paragraph

Two column

Column A

Paragraph
Paragraph

Column B

Paragraph
Table

Mixed layout

Heading
Paragraph

Table

Paragraph

Therefore, extraction must consider the **physical position of content on the page**.

Where available, we preserve information such as:

- bounding box
- page coordinates
- element position
- page dimensions

This helps reconstruct the correct reading order.

10. Stage 4 — Reading Order Reconstruction

A major risk is that raw PDF extraction may return text in an order that doesn't match how a human reads the page.

For example:

Expected:

Column 1

A

B

C

Column 2

D

E

F

But naive extraction could produce:

A

D

B

E

C

F

That would corrupt the meaning before chunking even begins.

Therefore, the extraction layer must reconstruct the most likely reading order using the page layout.

11. Stage 5 — Element Detection

We classify extracted content into structural elements such as:

heading
subheading
paragraph
list
clause
table
header
footer
image

For example:

```
{
  "page_number": 20,
  "elements": [
    {
      "element_id": "p20_e01",
      "type": "heading",
      "text": "Waiting Period"
    },
    {
      "element_id": "p20_e02",
      "type": "paragraph",
      "text": "..."
    }
  ]
}
```

At this stage, **we are still not chunking**.

We are simply understanding what the PDF contains.

12. Stage 6 — Table Preservation

Tables require special treatment.

Suppose the policy contains:

Treatment	Limit	Waiting Period
Treatment A	₹5 lakh	30 days
Treatment B	₹10 lakh	24 months

We cannot safely flatten this into random text because the relationship between the columns matters.

Therefore, we preserve:

```
Table
├── Headers
├── Row 1
├── Row 2
└── Row 3
```

The later chunking layer can decide how the table should be represented for retrieval.

13. Stage 7 — Page Continuity

This is another reason we need the intermediate representation.

Suppose:

Page 20:
"The waiting period for the specified
condition shall..."

Page 21:

"...remain applicable for 24 months."

We should **not decide at extraction time** that these are two unrelated pieces.

Instead we preserve:

Page 20 → Element A
Page 21 → Element B

Then the later structure/chunking layer determines whether:

A + B

form one logical unit.

This separation makes the architecture cleaner.

14. Proposed Intermediate Representation

Each PDF will have its own structured representation.

Conceptually:

```
{
  "document": {
    "document_id": "STAR_COMP_V1",
    "insurer": "Star Health",
    "product": "Star Comprehensive",
    "uin": "SHAHLIP26044V092526"
  },
  "pages": [
    {
      "page_number": 20,
      "layout": {
        "type": "two_column"
      },
      "elements": [
        {
          "element_id": "p20_e01",
          "type": "heading",
          "text": "Waiting Period",
          "reading_order": 1,
          "bbox": [70, 100, 500, 130]
        }
      ]
    }
  ]
}
```

```

    },
    {
      "element_id": "p20_e02",
      "type": "paragraph",
      "text": "...",
      "reading_order": 2,
      "bbox": [70, 150, 500, 350]
    }
  ]
}
]
}

```

This is an **intermediate representation**, not our final RAG dataset.

15. Why We Store Each PDF Separately

We don't want:

`all_5_policies.json`

containing hundreds of pages of mixed content.

Instead:

```

structured/
├── HDFC_ERGO/
│   └── document.json
├── NIVA_BUPA/
│   └── document.json
├── CARE_HEALTH/
│   └── document.json
├── ICICI_LOMBARD/
│   └── document.json
└── STAR_HEALTH/
    └── document.json

```

Advantages:

- independent processing
- easier debugging
- easier versioning
- easier reprocessing
- prevents accidental cross-product mixing
- preserves document identity

The corpus registry connects the five documents.

16. What We Will NOT Do During Extraction

This distinction should be very clear to the team.

We will NOT:

- ✗ summarize the PDF
- ✗ ask an LLM to rewrite the policy
- ✗ merge all PDFs into one text
- ✗ split every page into a chunk
- ✗ split every 500 tokens blindly
- ✗ discard page information
- ✗ flatten tables without structure
- ✗ assume all PDFs have the same layout

We WILL:

- ✓ preserve the original PDFs
- ✓ extract page-level content
- ✓ preserve layout information
- ✓ identify document elements
- ✓ preserve tables
- ✓ preserve source locations
- ✓ preserve document identity
- ✓ create an intermediate structured representation

17. Why This Layer Is Important for the Entire RAG

Everything downstream depends on this layer.

If extraction is wrong:

```
Wrong extraction
  ↓
Wrong structure
  ↓
Wrong chunks
  ↓
```

Wrong embeddings
↓
Wrong retrieval
↓
Wrong context
↓
Wrong answer

Therefore:

A high-quality RAG system begins with a high-quality document representation.

We don't want to hide extraction errors inside the vector database.

18. Where LangChain, FAISS and Chroma Come In

These technologies are important, but **not at the extraction stage**.

Our architecture is:

5 PDFs
↓
PDF extraction
↓
Structured document representation
↓
Smart chunking
↓
Validated chunks
↓
LangChain Document abstraction
↓
Embedding model
↓
FAISS / Chroma
↓
Retriever
↓
Reranker
↓
LLM

So the team should understand:

PDF extraction

Preserves the source.

Chunking

Creates meaningful retrieval units.

LangChain

Orchestrates document/retrieval workflows.

FAISS / Chroma

Store/search vector representations.

LLM

Generates the final response from retrieved evidence.

19. Why We Are Designing Extraction Before Chunking

Our current decision is:

Do not start smart chunking until the intermediate document representation is defined and validated.

Because we first need to know:

`What did the PDF actually contain?`

Only then can we ask:

`Which pieces belong together?`

That is the difference between:

Extraction

and

Semantic chunking.

20. What We Need to Agree on as a Team

Before implementation, the team should finalize:

Extraction

- Which PDF extraction library?
- How will layout be detected?
- How will tables be extracted?
- How will scanned pages be handled?
- How will reading order be reconstructed?

Intermediate schema

- What fields are mandatory?
- What element types do we support?
- How do we represent tables?
- How do we represent coordinates?
- How do we preserve page references?

Validation

- How do we detect extraction failures?
- How do we measure extraction completeness?
- How do we identify suspicious pages?

Storage

```
Raw PDFs
↓
Structured JSON
↓
Chunk JSON/JSONL
↓
Validation records
↓
Vector store
```

Only after these are finalized should we implement smart chunking.